ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

		Скобцов Ю.А.
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 7

Оптимизация функций многих переменных с помощью роевых алгоритмов
Вариант 4
по курсу: Эволюционные методы проектирования программно-информационных систем

РАБОТУ ВЫПОЛНИЛ
СТУДЕНТ

гр. №	4332		Могилатов С.И.
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1 ЗАДАНИЕ И ПОСТАНОВКА ЗАДАЧИ 4

1.1 Индивидуальное задание 4

1.2 Математическая модель 4

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 5

2.1 Глобальный роевой алгоритм 5

2.2 Границы и критерий останова 5

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И ПЛАН ЭКСПЕРИМЕНТА 6

3.1 Последовательность работы программы 6

3.2 Настройки серий 6

4 РЕЗУЛЬТАТЫ 7

4.1 Влияние размерности 9

4.2 Влияние размера роя 9

4.3 Влияние инерции 10

4.4 Критерий остановки 11

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ 13

ЗАКЛЮЧЕНИЕ 15

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 16

ПРИЛОЖЕНИЕ А 17

Модуль глобального роевого алгоритма 17

ПРИЛОЖЕНИЕ Б 19

Сценарий экспериментов и построения графиков 19

ВВЕДЕНИЕ

Роевой алгоритм частиц (PSO) ищет экстремум непрерывной функции, перемещая набор точек-пробных решений. Скорость каждой частицы зависит от её прежнего движения, лучшей собственной позиции и лучшей найденной роем позиции. В лабораторной работе этот метод применяется к сдвинутой осепараллельной гиперэллипсоидной функции варианта 4.

Цель работы — реализовать глобальный PSO, показать поверхность двумерной функции и движение частиц по поколениям, исследовать влияние размерности, числа частиц, коэффициента инерции и критерия остановки.

Отчёт содержит постановку задачи, описание алгоритма, воспроизводимые параметры и исходный код, результаты пяти повторов для n=2, 3, 5 и 10, графики и ответы на контрольные вопросы. Правила оформления проверены по актуальным требованиям ГУАП [1].

1 ЗАДАНИЕ И ПОСТАНОВКА ЗАДАЧИ

1.1 Индивидуальное задание

Вариант 4 — Moved axis parallel hyper-ellipsoid function. В таблице указаны глобальный минимум f(x)=0 и координаты x_i=5i для i=1,…,n. Практическая часть требует построить график функции и показать популяцию при n=2, предусмотреть пошаговый просмотр, исследовать влияние размера популяции и критерия останова, а также сравнить поиск для n=3, 5 и 10 [2].

В методичке напечатана формула $\Sigma 5i \cdot x_i^2$, которая минимальна в нуле, хотя рядом заявлена точка $x_i=5i$ и функция названа сдвинутой. Чтобы согласовать название и объявленный минимум, в расчётах используется $f_n(x)=\Sigma_{i=1}^n 5i(x_i-5i)^2$. При таком определении $f_n(x^*)=0$ в $x^*=(5,10,\ldots,5n)$. Этот выбор трактовки явно фиксируется, поскольку исходная строка задания противоречива.

Методичка не указывает границы поиска для этой строки. В программе принят общий диапазон $x_i \in [-55,55]$ для всех координат: он включает целевые значения до 50 при n=10 и оставляет оптимум внутри области. Это экспериментальное допущение, а не ограничение из таблицы задания.

1.2 Математическая модель

Для каждого числа измерений n целевая функция задаётся суммой независимых квадратичных слагаемых с положительными весами: $f_n(x)=\Sigma_{i=1}^n 5i(x_i-5i)^2$. Производная по координате равна $\partial f/\partial x_i=10 \cdot 5i(x_i-5i)$; поэтому единственная стационарная точка — $x_i=5i$. Все веса положительны, матрица Гессе диагональна и положительно определена, следовательно эта точка является единственным глобальным минимумом с $f^*=0$.

Для двумерного случая $f_2(x_1,x_2)=5(x_1-5)^2+10(x_2-10)^2$. Линии уровня — эллипсы с центром в (5,10).

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Глобальный роевой алгоритм

В PSO каждая частица i имеет положение x_i(t), скорость v_i(t), личный лучший результат p_i и вектор глобально лучшей позиции g. После оценки функции память частицы обновляется, если новое положение лучше прежнего. Затем скорость и позиция рассчитываются по формулам [2; 3]:

$$v_i(t+1)=w\cdot v_i(t)+c_1\cdot r_1\cdot(p_i-x_i(t))+c_2\cdot r_2\cdot(g-x_i(t));$$

$$x_i(t+1)=x_i(t)+v_i(t+1).$$

Здесь w — инерционный вес; c_1 и c_2 — когнитивный и социальный коэффициенты; r_1 и r_2 — независимые случайные числа из [0,1]. Когнитивная часть направляет частицу к её собственному историческому успеху, социальная — к лучшему решению роя. Обмен информацией со всеми частицами образует глобальную структуру соседства.

Исходная формула методички не содержит множитель инерции, то есть соответствует w=1, и рекомендует $c_1=c_2=2$. В программе применяется инерционная модификация PSO с w=0,7298 и $c_1=c_2=1{,}49618$ [4]. Это делает явным контроль переноса скорости; вес w отдельно исследуется. Для случайных множителей используются равномерные значения [0,1], как в канонической записи PSO. Фраза методички о нормальном распределении при одновременно указанном интервале [0,1] неоднозначна.

2.2 Границы и критерий останова

Скорость ограничивается по модулю 20% ширины области, чтобы отдельный шаг не переносил частицу через весь диапазон. При выходе за границу применяется отражение и частичное гашение скорости. Остановка базового запуска наступает при $f_best \le 10^{-8}$, при отсутствии улучшения 100 поколений либо после 1000 поколений. При исследовании patience порог f отключается, чтобы изолировать действие стагнационного критерия.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И ПЛАН ЭКСПЕРИМЕНТА

3.1 Последовательность работы программы

1. Создать воспроизводимый генератор случайных чисел по заданному seed и равномерно инициализировать положения и скорости частиц.
2. Вычислить f(x) для всех частиц; сохранить их личные лучшие позиции и выбрать глобально лучшую.
3. На каждом поколении вычислить новую скорость с инерционной, когнитивной и социальной составляющими.
4. Ограничить скорость, отразить координаты у границ, пересчитать функцию и обновить личный и глобальный рекорды.
5. Записать лучшее и среднее значение функции; проверить точность, стагнацию и возраст популяции.
6. Сохранить CSV/JSON, график поверхности с популяцией, снимки поиска, кривые сходимости и диаграммы параметров.

Модуль алгоритма находится в приложении А, сценарий запусков и построения графиков — в приложении Б. Программа поддерживает краткую сводку и режим --verbose с печатью истории демонстрационного запуска.

3.2 Настройки серий

Демонстрационный запуск: n=2, 60 частиц, w=0,7298, $c_1=c_2=1{,}49618$, seed=247, точность 10^{-8} и patience=100. Сравнения размерностей и параметров повторяются на пяти seed; за раз меняется один фактор.

Таблица 2 — Настройки экспериментов

Серия	Параметр	Значения	Повторов
Размерность	n	2, 3, 5, 10	5 на n
Размер роя	число частиц	20, 40, 80, 120	5 на значение
Инерция	w	0,4; 0,7298; 0,9	5 на значение
Остановка	patience	30, 60, 120; f-порог отключён	5 на значение
Общие	$c_1=c_2$; диапазон	1,49618; [−55,55]	фиксированы

Мутация к PSO неприменима: алгоритм не использует генетических операторов. Вместо её вероятности исследуется инерционный вес w, напрямую влияющий на движение частиц.

4 РЕЗУЛЬТАТЫ

Для n=2 при seed=247 найдено x=(5.00001629; 9.99999437) и f=1.64e-09. Точное решение — (5; 10), f*=0; евклидова ошибка координат равна 1.72e-05. Порог $f \le 10^{-8}$ достигнут за 63 поколений; вычислительное время составило 0.0080 с.

Рисунок 1 показывает поверхность двумерной целевой функции и положение частиц последнего поколения. Для точного чтения области поиска линии уровня и числовые оси дополнены на рисунке 2; снимки поколений показывают стягивание роя к центру эллиптической впадины.

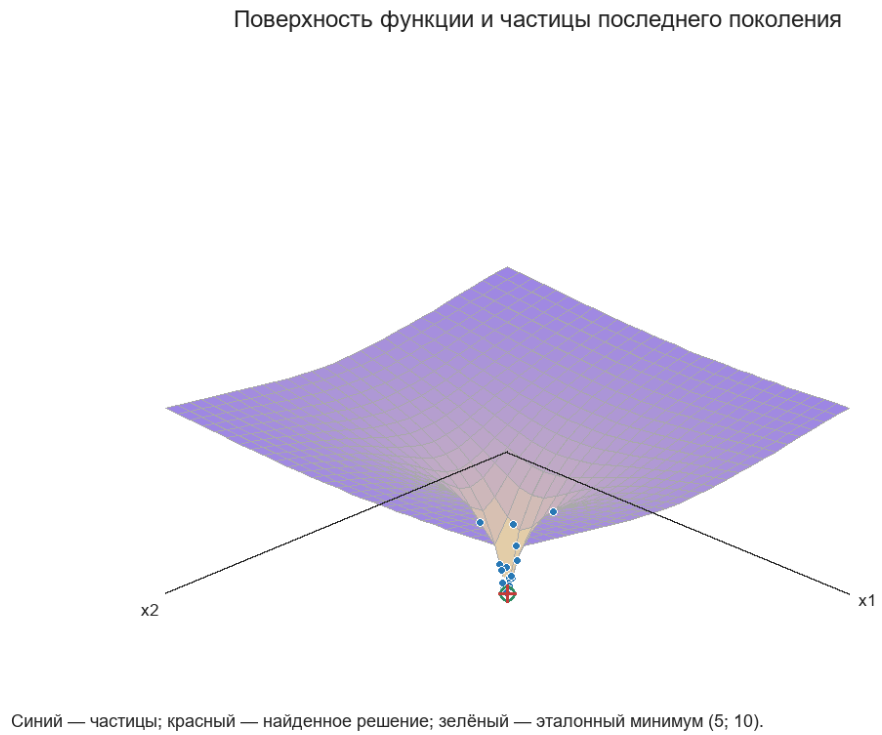


Рисунок 1 — Поверхность функции и частицы последнего поколения

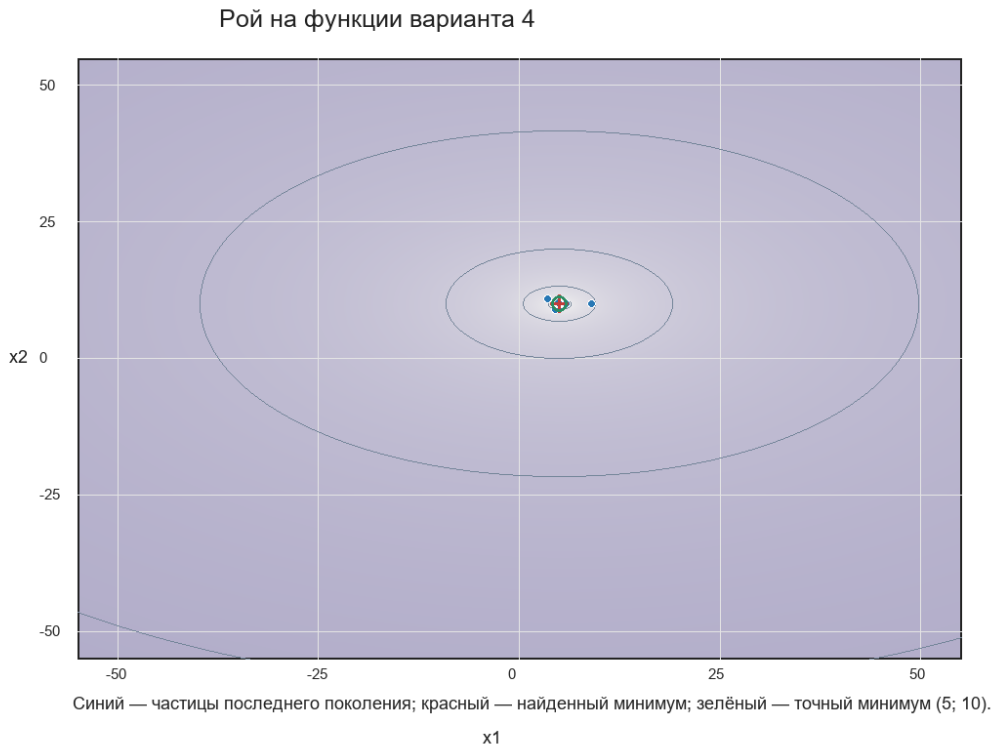


Рисунок 2 — Линии уровня функции и найденный минимум

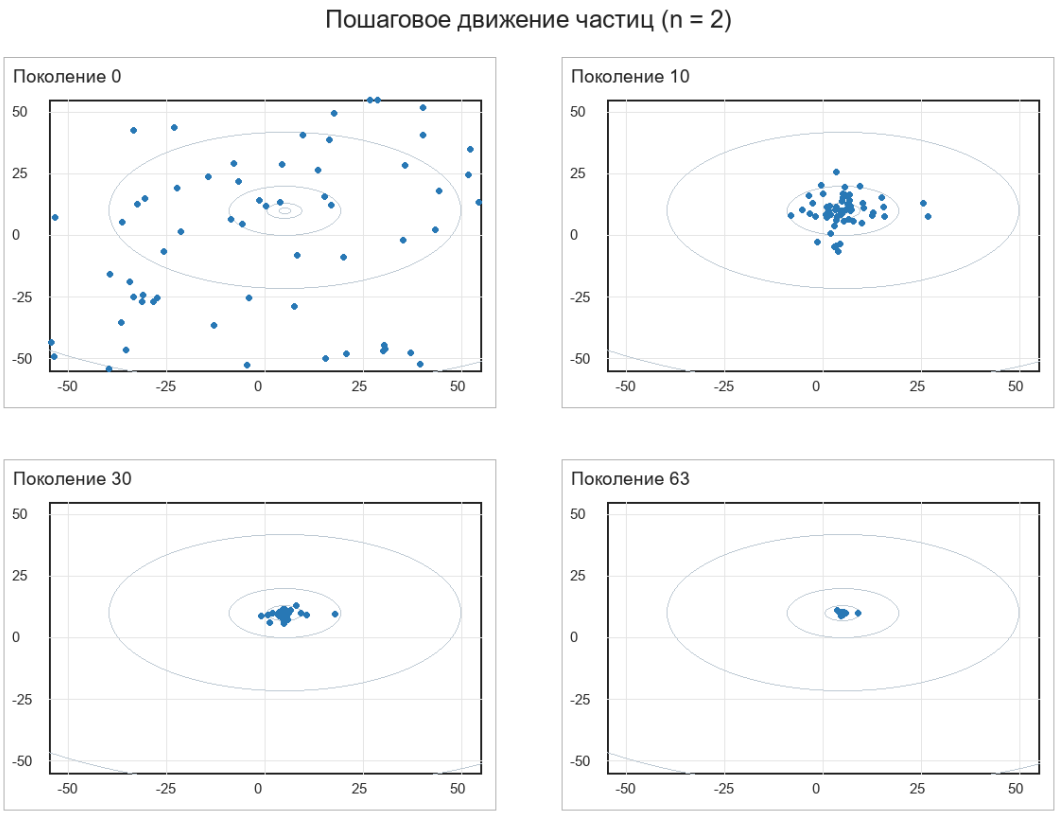


Рисунок 3 — Пошаговое сближение частиц с минимумом

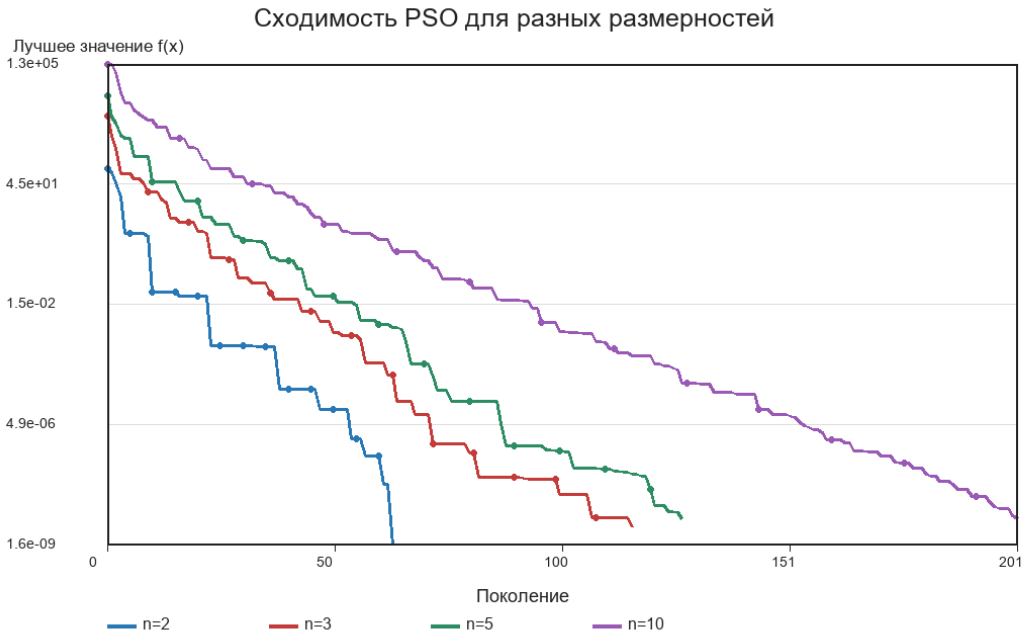


Рисунок 4 — Сходимость для n=2, 3, 5 и 10

4.1 Влияние размерности

Сводные средние значения и ошибки координат для четырёх размерностей приведены в таблице 3.

Таблица 3 — Средние результаты по пяти запускам

n	Среднее f best	Ошибка координат	Поколения	Время, с
2	4.64e-09	2.39e-05	76.6	0.0103
3	5.97e-09	2.68e-05	107.8	0.0203
5	7.81e-09	2.60e-05	135.8	0.0395
10	8.12e-09	1.97e-05	206.4	0.1028

Во всех двадцати измерениях алгоритм достиг порога 10^{-8}. При росте числа координат средние поколения и время увеличились: у более размерной задачи выше число степеней свободы и сложнее одновременно сократить ошибку каждой координаты. Времена отражают локальные короткие запуски и зависят от состояния компьютера; сравнение поколений устойчивее сравнения десятых долей миллисекунды.

4.2 Влияние размера роя

Сравнение четырёх размеров роя по пяти независимым запускам сведено в таблицу 4.

Таблица 4 — Размер роя, пять повторов

Частиц	Среднее f best ± SD	Поколения	Время, с	Порог достигнут

Частиц	Среднее f best ± SD	Поколения	Время, с	Порог достигнут
20	2.82e-09 ± 3.9e-09	90.4	0.0040	5/5
40	5.07e-09 ± 1.4e-09	86.4	0.0074	5/5
80	7.05e-09 ± 3.7e-09	77.8	0.0156	5/5
120	6.06e-09 ± 3.1e-09	73.4	0.0238	5/5

Зависимость среднего лучшего значения от размера роя показана на рисунке 5.

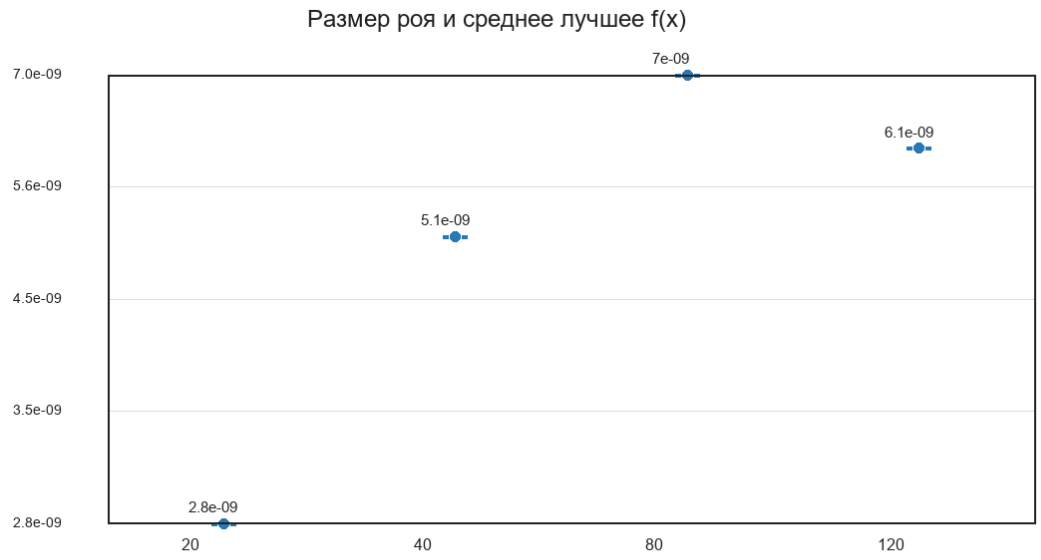


Рисунок 5 — Среднее лучшее значение при разном размере роя, логарифмическая шкала

Порог точности достигнут во всех сериях. Увеличение числа частиц сократило среднее число поколений примерно с 90 до 73, но увеличило время одного поиска с примерно 0,004 до 0,018 с. Лучшее среднее f не уменьшается монотонно с числом частиц, что ожидаемо для конечной выборки случайных запусков; по пяти повторам нельзя заключить, что конкретный размер всегда точнее.

4.3 Влияние инерции

Сводка результатов для трёх значений инерции представлена в таблице 5.

Таблица 5 — Коэффициент инерции, пять повторов

w	Среднее f best ± SD	Поколения	Время, с	Порог достигнут
0.4	4.19e-09 ± 3.0e-09	31.2	0.0048	5/5
0.7298	3.48e-09 ± 3.1e-09	84.2	0.0114	5/5
0.9	9.53e-04 ± 1.1e-03	242.0	0.0324	1/5

График среднего лучшего значения при изменении w приведён на рисунке 6.

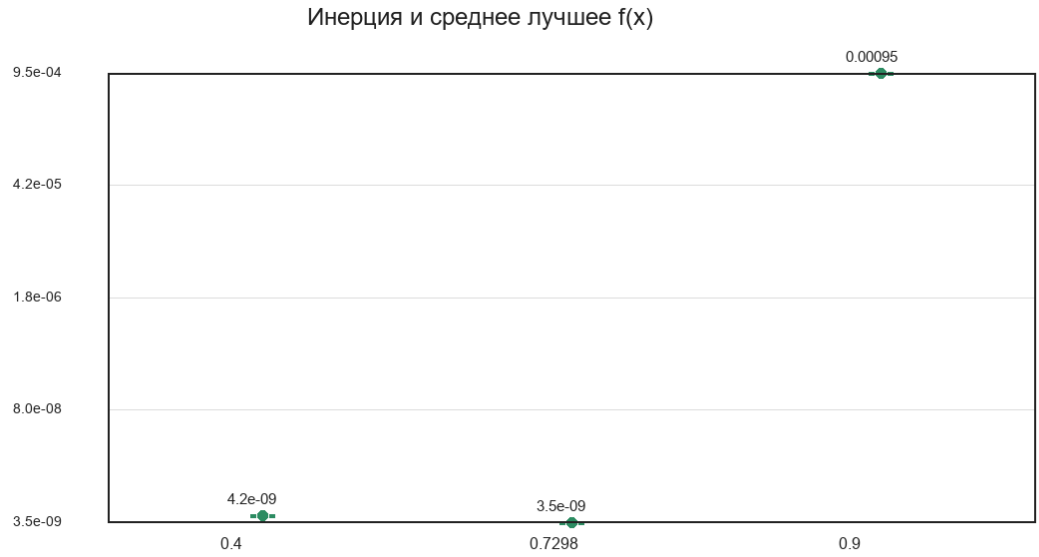


Рисунок 6 — Среднее лучшее значение при разных w, логарифмическая шкала

В испытанных сериях w=0,4 и w=0,7298 достигли точности во всех пяти повторах. При w=0,9 порог достигнут только в одном повторе, а среднее лучшее значение заметно выше. На этой задаче чрезмерная инерция сохраняет большой шаг и затрудняет точное оседание роя у минимума. Это наблюдение относится к выбранным коэффициентам и диапазону, а не является универсальным правилом для всех PSO-задач.

4.4 Критерий остановки

Сравнение трёх интервалов стагнации приведено в таблице 6.

Таблица 6 — Интервал стагнации без порога f

Поколений без улучшения	Среднее f best ± SD	Поколения	Время, с
30	7.29e-17 ± 9.0e-17	160.0	0.0202
60	1.58e-19 ± 2.3e-19	199.0	0.0269
120	1.95e-25 ± 3.4e-25	271.4	0.0355

Зависимость числа поколений до останова от patience показана на рисунке 7.

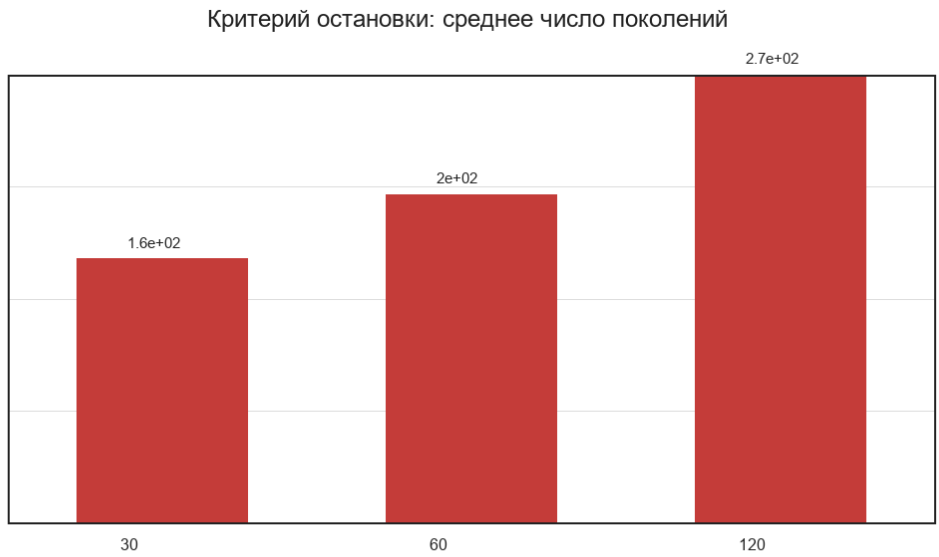


Рисунок 7 — Среднее число поколений до остановки

При отключённом пороге по целевой функции больший patience позволил продолжить поиск: среднее число поколений выросло с 160 при patience=30 до 271 при patience=120. Среднее лучшее f стало меньше, однако расчёт также занял больше времени. Это показывает компромисс между ранней остановкой и дополнительным доуточнением решения. Пять повторов дают иллюстративную, а не статистически исчерпывающую оценку.

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Потенциальное решение представляется вектором вещественных координат частицы $x_i=(x_{i1},\ldots,x_{in})$.

2. Координаты обновляются прибавлением новой скорости: $x_i(t+1)=x_i(t)+v_i(t+1)$. Скорость включает прежнюю скорость, притяжение к личному лучшему и к глобально лучшему вектору.

3. Когнитивная составляющая $c_1r_1(p_i-x_i)$ возвращает частицу в сторону её собственной лучшей ранее найденной позиции.

4. Социальная составляющая $c_2r_2(g-x_i)$ направляет частицу к лучшему решению, найденному всем роем или её соседями.

5. Глобальный PSO инициализирует позиции и скорости, оценивает целевую функцию, обновляет личный и глобальный рекорды, перемещает частицы и повторяет цикл до критерия останова.

6. В глобальном PSO каждая частица использует лучшее решение всего роя; в локальном — лучшее решение ограниченного соседства. Локальный вариант распространяет информацию медленнее и может поддерживать разнообразие.

7. Социальная структура задаёт, от каких соседей частица получает информацию. Примеры — глобальная звезда, кольцо, решётка и иные графы соседства.

8. Локальный PSO задаёт окрестность каждой частицы, выбирает лучшего соседа и использует его позицию в социальной части формулы скорости вместо единого глобального рекорда.

9. Личная лучшая позиция p_i — положение, в котором частица i достигла наименьшего значения функции среди всех её предыдущих посещений.

10. Глобальная лучшая позиция g — лучшая из личных лучших позиций всех частиц роя на текущем шаге.

11. К основным аспектам относятся кодирование решения координатами, инициализация роя, оценка качества, память p_i и g, обновление скорости и положения, обработка границ и критерий остановки.

12. Обычно используют максимальное число поколений, достижение целевой точности, ограничение времени или отсутствие улучшения в течение заданного числа поколений.

13. Положения обычно случайно и независимо выбираются внутри области поиска; начальные скорости задаются случайно в допустимом диапазоне. Для воспроизводимости фиксируют seed.

14. Основные параметры: размер роя, инерционный вес w, когнитивный c_1 и социальный c_2 коэффициенты, диапазон скоростей, границы переменных и критерий остановки.

15. Известны глобальные и локальные схемы, PSO с инерционным весом или коэффициентом сжатия, адаптивные и самоорганизующиеся варианты, а также дискретные и гибридные алгоритмы.

16. PSO, эволюционные стратегии (ЭС) и генетические алгоритмы (ГА) — популяционные стохастические методы оптимизации. Они итеративно оценивают множество кандидатов и используют их качество для формирования следующих решений.

17. PSO перемещает векторы решений с помощью скоростей и социальной памяти. ЭС изменяют непрерывные параметры мутацией и отбором родителей/потомков. ГА кодирует хромосомы и применяет отбор, кроссовер и мутацию. В базовом PSO генетических операторов нет.

ЗАКЛЮЧЕНИЕ

Реализован глобальный роевой алгоритм для оптимизации функции варианта 4. В соответствии с объявленным в методичке минимумом исследовалась сдвинутая функция $f_n(x)=\Sigma 5i(x_i-5i)^2$. Для n=2 найдено решение с $f=1{,}64\cdot10^{-9}$ и координатной ошибкой порядка 10^{-5}; порог 10^{-8} достигнут за 63 поколения.

Во всех пяти повторах для каждой размерности n=2, 3, 5, 10 достигнут заданный порог. С увеличением размерности возросли средние поколения и время. Более крупный рой выполнял меньше поколений, но увеличивал время из-за большего числа вычислений на поколение. Инерция w=0,9 оказалась неудачной для выбранной задачи, а увеличение patience при отключённом целевом пороге давало более точные решения ценой дополнительного времени.

В исходной таблице задания есть противоречие между формулой и заявленной точкой минимума; кроме того, методичка просит исследовать вероятность мутации, которая относится к генетическому алгоритму, а не PSO. В отчёте приняты явные, проверяемые трактовки и представлены код, исходные данные запусков, графики и шаги поиска.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГУАП. Для учебного процесса: нормативные требования к оформлению студенческих работ и бланки титульных листов лабораторных работ. URL: https://guap.ru/c/regdocs/docs/uch (дата обращения: 10.11.2026).

2. Методические указания. Лабораторная работа № 7 «Оптимизация функций многих переменных с помощью роевых алгоритмов». Разделы 7.1–7.2 и приложение 1. Предоставленный учебный материал.

3. Kennedy J., Eberhart R. Particle swarm optimization // Proceedings of ICNN’95 — International Conference on Neural Networks. 1995. Vol. 4. P. 1942–1948. DOI: 10.1109/ICNN.1995.488968.

4. Shi Y., Eberhart R. A modified particle swarm optimizer // Proceedings of the IEEE International Conference on Evolutionary Computation. Anchorage, 1998. P. 69–73. DOI: 10.1109/ICEC.1998.699146.

ПРИЛОЖЕНИЕ А

Модуль глобального роевого алгоритма

```
"""Global-best particle swarm optimizer for the GUAP lab 7 benchmark."""


from __future__ import annotations


from dataclasses import dataclass
import random
import time
from typing import Callable




def moved_ellipsoid(position: list[float]) -> float:
    """Variant 4: sum(5*i*(x_i-5*i)^2), with i numbered from one."""
    return sum(5.0 * i * (x - 5.0 * i) ** 2 for i, x in enumerate(position, 1))




@dataclass
class SwarmResult:
    best_position: list[float]
    best_value: float
    best_history: list[float]
    mean_history: list[float]
    snapshots: dict[int, list[list[float]]]
    elapsed_seconds: float
    generations: int
    stop_reason: str




class ParticleSwarm:
    """Global-best PSO with inertia, bounded velocity, and reflective limits."""


    def __init__(
        self,
        dimensions: int,
        *,
        particles: int = 60,
        max_generations: int = 1000,
        patience: int = 100,
        tolerance: float = 1e-8,
        inertia: float = 0.7298,
        cognitive: float = 1.49618,
        social: float = 1.49618,
        lower: float = -55.0,
        upper: float = 55.0,
        seed: int = 2026,
        objective: Callable[[list[float]], float] = moved_ellipsoid,
    ) -> None:
        if dimensions < 2:
            raise ValueError("dimensions must be at least 2")
        if particles < 2 or max_generations < 1 or patience < 1:
            raise ValueError("particles, generations, and patience must be positive")
        if lower >= upper:
            raise ValueError("lower bound must be less than upper bound")
        self.dimensions = dimensions
        self.particles = particles
        self.max_generations = max_generations
        self.patience = patience
        self.tolerance = tolerance
        self.inertia = inertia
        self.cognitive = cognitive
        self.social = social
        self.lower = lower
        self.upper = upper
        self.seed = seed
        self.objective = objective
        self.rng = random.Random(seed)


    def run(self, *, capture: tuple[int, ...] = ()) -> SwarmResult:
        started = time.perf_counter()
        span = self.upper - self.lower
        velocity_limit = 0.2 * span
        positions = [
```

```
    [self.rng.uniform(self.lower, self.upper) for _ in range(self.dimensions)]
    for _ in range(self.particles)
]
velocities = [
    [self.rng.uniform(-0.1 * span, 0.1 * span) for _ in range(self.dimensions)]
    for _ in range(self.particles)
]
values = [self.objective(point) for point in positions]
personal_positions = [point[:] for point in positions]
personal_values = values[:]
best_index = min(range(self.particles), key=personal_values.__getitem__)
global_position = personal_positions[best_index][:]
global_value = personal_values[best_index]
best_history = [global_value]
mean_history = [sum(values) / len(values)]
snapshots: dict[int, list[list[float]]] = {0: [point[:] for point in positions]}
stale = 0
stop_reason = "достигнут предел поколений"
generations = 0


for generation in range(1, self.max_generations + 1):
    old_best = global_value
    for p in range(self.particles):
        for d in range(self.dimensions):
            r1 = self.rng.random()
            r2 = self.rng.random()
            velocities[p][d] = (
                self.inertia * velocities[p][d]
                + self.cognitive * r1 * (personal_positions[p][d] - positions[p][d])
                + self.social * r2 * (global_position[d] - positions[p][d])
            )
            velocities[p][d] = max(-velocity_limit, min(velocity_limit, velocities[p][d]))
            candidate = positions[p][d] + velocities[p][d]
            if candidate < self.lower:
                candidate = self.lower + (self.lower - candidate)
                velocities[p][d] *= -0.5
            elif candidate > self.upper:
                candidate = self.upper - (candidate - self.upper)
                velocities[p][d] *= -0.5
            positions[p][d] = max(self.lower, min(self.upper, candidate))


        values[p] = self.objective(positions[p])
        if values[p] < personal_values[p]:
            personal_values[p] = values[p]
            personal_positions[p] = positions[p][:]
            if values[p] < global_value:
                global_value = values[p]
                global_position = positions[p][:]


    generations = generation
    best_history.append(global_value)
    mean_history.append(sum(values) / len(values))
    if generation in capture:
        snapshots[generation] = [point[:] for point in positions]
    if global_value <= self.tolerance:
        stop_reason = f"достигнута точность f ≤ {self.tolerance:g}"
        break
    if global_value < old_best - max(1e-14, abs(old_best) * 1e-12):
        stale = 0
    else:
        stale += 1
    if stale >= self.patience:
        stop_reason = f"нет улучшения {self.patience} поколений"
        break


if generations not in snapshots:
    snapshots[generations] = [point[:] for point in positions]
return SwarmResult(
    best_position=global_position,
    best_value=global_value,
    best_history=best_history,
    mean_history=mean_history,
    snapshots=snapshots,
    elapsed_seconds=time.perf_counter() - started,
    generations=generations,
    stop_reason=stop_reason,
)
```

ПРИЛОЖЕНИЕ Б

Сценарий экспериментов и построения графиков

```
"""Run reproducible PSO experiments and create report-ready figures."""


from __future__ import annotations


import argparse
import csv
import json
import math
from pathlib import Path
import statistics
import sys


from PIL import Image, ImageDraw, ImageFont


from pso import ParticleSwarm



ROOT = Path(__file__).resolve().parent
RESULTS = ROOT / "results"
RESULTS.mkdir(exist_ok=True)
FONT_PATH = r"C:\Windows\Fonts\arial.ttf"
BLUE = "#2878b5"
RED = "#c43c39"
GREEN = "#2c8c62"
BLACK = "#222222"



def font(size: int):
    return ImageFont.truetype(FONT_PATH, size)



def mean_sd(values):
    return statistics.mean(values), (statistics.stdev(values) if len(values) > 1 else 0.0)



def position_error(position):
    return math.sqrt(sum((x - 5 * i) ** 2 for i, x in enumerate(position, 1)))



def xy_map(x, y, box, limits=(-55.0, 55.0)):
    left, top, right, bottom = box
    low, high = limits
    px = left + (x - low) / (high - low) * (right - left)
    py = bottom - (y - low) / (high - low) * (bottom - top)
    return px, py



def draw_axes(draw, box, limits=(-55, 55), ticks=(-50, -25, 0, 25, 50)):
    left, top, right, bottom = box
    draw.rectangle(box, outline=BLACK, width=2)
    for tick in ticks:
        x, _ = xy_map(tick, limits[0], box, limits)
        _, y = xy_map(limits[0], tick, box, limits)
        draw.line((x, top, x, bottom), fill="#e5e5e5", width=1)
        draw.line((left, y, right, y), fill="#e5e5e5", width=1)
        draw.text((x - 12, bottom + 7), str(tick), fill=BLACK, font=font(15))
        draw.text((left - 39, y - 9), str(tick), fill=BLACK, font=font(15))



def draw_clipped_ellipse(image, box, bounds, color="#75869a", width=1):
    left, top, right, bottom = box
    overlay = Image.new("RGBA", (right-left+1, bottom-top+1), (0, 0, 0, 0))
    odraw = ImageDraw.Draw(overlay)
    x0, y0, x1, y1 = bounds
```

```
    odraw.ellipse((x0-left, y0-top, x1-left, y1-top), outline=color, width=width)
    image.paste(overlay, (left, top), overlay)



def draw_contour(path: Path, particles, best_position, title="Рой на функции варианта 4"):
    image = Image.new("RGB", (1100, 800), "white")
    draw = ImageDraw.Draw(image)
    draw.text((250, 20), title, fill=BLACK, font=font(25))
    box = (105, 75, 1010, 690)
    low, high = -55, 55
    # Log-scaled objective background keeps both the basin and the search range visible.
    for py in range(box[1], box[3]):
        y = high - (py - box[1]) / (box[3] - box[1]) * (high - low)
        for px in range(box[0], box[2]):
            x = low + (px - box[0]) / (box[2] - box[0]) * (high - low)
            value = 5 * (x - 5) ** 2 + 10 * (y - 10) ** 2
            shade = min(1.0, math.log10(value + 1) / math.log10(100000))
            image.putpixel((px, py), (int(250 - 75 * shade), int(250 - 80 * shade), int(245 - 45 * shade)))
    draw = ImageDraw.Draw(image)
    draw_axes(draw, box)
    # The exact objective contours are ellipses: 5(x1-5)^2 + 10(x2-10)^2 = level.
    for level in (10, 100, 1000, 10000, 50000):
        rx = math.sqrt(level / 5) / 110 * (box[2] - box[0])
        ry = math.sqrt(level / 10) / 110 * (box[3] - box[1])
        cx, cy = xy_map(5, 10, box)
        draw_clipped_ellipse(image, box, (cx-rx, cy-ry, cx+rx, cy+ry))
        draw = ImageDraw.Draw(image)
    for x, y in particles:
        px, py = xy_map(x, y, box)
        draw.ellipse((px-4, py-4, px+4, py+4), fill=BLUE, outline="white", width=1)
    bx, by = xy_map(best_position[0], best_position[1], box)
    ox, oy = xy_map(5, 10, box)
    draw.line((bx-9, by, bx+9, by), fill=RED, width=3)
    draw.line((bx, by-9, bx, by+9), fill=RED, width=3)
    draw.ellipse((ox-8, oy-8, ox+8, oy+8), outline=GREEN, width=3)
    draw.text((100, 725), "Синий — частицы последнего поколения; красный — найденный минимум; зелёный — точный минимум
(5; 10).", fill=BLACK, font=font(18))
    draw.text((520, 760), "x1", fill=BLACK, font=font(18))
    draw.text((35, 370), "x2", fill=BLACK, font=font(18))
    image.save(path)



def draw_surface(path: Path, particles, best_position):
    """Draw a perspective surface and the final two-dimensional swarm with Pillow."""
    image=Image.new("RGB",(1100,800),"white"); draw=ImageDraw.Draw(image)
    draw.text((270,18),"Поверхность функции и частицы последнего поколения",fill=BLACK,font=font(24))
    xmin,xmax=-10.0,20.0; ymin,ymax=-5.0,25.0
    nx=ny=28
    def objective(x,y): return 5*(x-5)**2+10*(y-10)**2
    zmax=objective(xmin,ymin)
    def project(x,y,z):
        u=(x-xmin)/(xmax-xmin); v=(y-ymin)/(ymax-ymin)
        height=190*math.log1p(z)/math.log1p(zmax)
        return 550+(u-v)*350, 475+(u+v)*145-height
    xs=[xmin+(xmax-xmin)*i/nx for i in range(nx+1)]
    ys=[ymin+(ymax-ymin)*j/ny for j in range(ny+1)]
    for order in range(nx+ny,-1,-1):
        for i in range(nx):
            j=order-i
            if not 0<=j<ny: continue
            x0,x1=xs[i],xs[i+1]; y0,y1=ys[j],ys[j+1]
            zvals=[objective(x0,y0),objective(x1,y0),objective(x1,y1),objective(x0,y1)]
            points=[project(x0,y0,zvals[0]),project(x1,y0,zvals[1]),project(x1,y1,zvals[2]),project(x0,y1,zvals[3])]
            shade=sum(math.log1p(z) for z in zvals)/4/math.log1p(zmax)
            color=(int(248-92*shade),int(226-93*shade),int(150+80*shade))
            draw.polygon(points,fill=color,outline="#a9a9a9")
    for x,y in particles:
        if xmin<=x<=xmax and ymin<=y<=ymax:
            px,py=project(x,y,objective(x,y)); draw.ellipse((px-4,py-4,px+4,py+4),fill=BLUE,outline="white")
    ox,oy=project(5,10,0)
    bx,by=project(best_position[0],best_position[1],objective(*best_position))
    draw.ellipse((ox-8,oy-8,ox+8,oy+8),outline=GREEN,width=3)
    draw.line((bx-9,by,bx+9,by),fill=RED,width=3);draw.line((bx,by-9,bx,by+9),fill=RED,width=3)
    # Base axes and labels.
    p00=project(xmin,ymin,0); p10=project(xmax,ymin,0); p01=project(xmin,ymax,0)
    draw.line((p00,p10),fill=BLACK,width=2);draw.line((p00,p01),fill=BLACK,width=2)
    draw.text((p10[0]+10,p10[1]-3),"x1",fill=BLACK,font=font(17))
    draw.text((p01[0]-25,p01[1]+7),"x2",fill=BLACK,font=font(17))
    draw.text((42,740),"Синий — частицы; красный — найденное решение; зелёный — эталонный минимум (5;
10).",fill=BLACK,font=font(18))
    image.save(path)
```

```
def draw_snapshots(result, path: Path, chosen):
    image = Image.new("RGB", (1200, 900), "white")
    draw = ImageDraw.Draw(image)
    draw.text((385, 20), "Пошаговое движение частиц (n = 2)", fill=BLACK, font=font(26))
    panels = [(45, 75, 565, 445), (635, 75, 1155, 445), (45, 500, 565, 870), (635, 500, 1155, 870)]
    for generation, panel in zip(chosen, panels):
        left, top, right, bottom = panel
        box = (left+48, top+45, right-15, bottom-38)
        draw.rectangle(panel, outline="#b0b0b0", width=1)
        draw.text((left+10, top+9), f"Поколение {generation}", fill=BLACK, font=font(19))
        # Elliptical objective contours centered at (5,10).
        draw_axes(draw, box)
        cx, cy = xy_map(5, 10, box)
        for level in (10, 100, 1000, 10000, 50000):
            rx = math.sqrt(level/5)/110*(box[2]-box[0])
            ry = math.sqrt(level/10)/110*(box[3]-box[1])
            draw_clipped_ellipse(image, box, (cx-rx,cy-ry,cx+rx,cy+ry), color="#bdc9d3")
            draw = ImageDraw.Draw(image)
        points = result.snapshots.get(generation, result.snapshots[max(result.snapshots)])
        for point in points:
            x,y=xy_map(point[0],point[1],box)
            draw.ellipse((x-3,y-3,x+3,y+3),fill=BLUE)
    image.save(path)



def draw_line_chart(path: Path, title: str, series, *, y_label: str, x_label: str, log_y=False):
    image = Image.new("RGB", (1100, 680), "white")
    draw = ImageDraw.Draw(image)
    draw.text((260, 20), title, fill=BLACK, font=font(25))
    box=(115,80,1015,555)
    all_y=[max(v,1e-15) for _, ys, _ in series for v in ys]
    y0=min(all_y); y1=max(all_y)
    if log_y:
        lo=math.log10(y0); hi=math.log10(y1)
        if hi-lo < .2: lo-=.1; hi+=.1
        convert=lambda v:(math.log10(max(v,1e-15))-lo)/(hi-lo)
        labels=[10**(lo+(hi-lo)*i/4) for i in range(5)]
        fmt=lambda v:f"{v:.1e}"
    else:
        if y1-y0 < 1e-10: y1=y0+1
        lo=y0; hi=y1
        convert=lambda v:(v-lo)/(hi-lo)
        labels=[lo+(hi-lo)*i/4 for i in range(5)]
        fmt=lambda v:f"{v:.3g}"
    for i,label in enumerate(labels):
        y=box[3]-(box[3]-box[1])*i/4
        draw.line((box[0],y,box[2],y),fill="#dddddd",width=1)
        draw.text((15,y-9),fmt(label),fill=BLACK,font=font(14))
    x_max=max(len(vals) for _,vals,_ in series)
    for name,values,color in series:
        if not values: continue
        points=[]
        for i,value in enumerate(values):
            x=box[0]+(box[2]-box[0])*i/max(1,x_max-1)
            y=box[3]-(box[3]-box[1])*convert(value)
            points.append((x,y))
        if len(points)>1: draw.line(points,fill=color,width=3)
        for x,y in points[::max(1,len(points)//12)]: draw.ellipse((x-3,y-3,x+3,y+3),fill=color)
    draw.rectangle(box,outline=BLACK,width=2)
    for tick in range(5):
        fraction=tick/4
        x=box[0]+(box[2]-box[0])*fraction
        draw.line((x,box[3],x,box[3]+6),fill=BLACK,width=1)
        draw.text((x-18,box[3]+9),str(round((x_max-1)*fraction)),fill=BLACK,font=font(14))
    draw.text((480,595),x_label,fill=BLACK,font=font(18))
    draw.text((22,52),y_label,fill=BLACK,font=font(17))
    legend_x=box[0]
    for name,_,color in series:
        draw.line((legend_x,635,legend_x+28,635),fill=color,width=4)
        draw.text((legend_x+35,624),name,fill=BLACK,font=font(16))
        legend_x+=max(160, len(name)*10)
    image.save(path)



def draw_bars(path: Path, title: str, rows, labels, value_key, color=BLUE, log=False):
    image=Image.new("RGB",(1100,650),"white"); draw=ImageDraw.Draw(image)
    draw.text((310,20),title,fill=BLACK,font=font(24))
    left,top,right,bottom=110,90,1040,540
    vals=[max(float(row[value_key]),1e-15) for row in rows]
    vmax=max(vals)
    vmin=min(vals) if log else 0
    if log:
        lo=math.log10(vmin); hi=math.log10(vmax)
        for k in range(5):
            frac=k/4; y=bottom-(bottom-top)*frac
```

```
            draw.line((left,y,right,y),fill="#dddddd",width=1)
            draw.text((15,y-9),f"{10**(lo+(hi-lo)*frac):.1e}",fill=BLACK,font=font(14))
    else:
        for k in range(5):
            y=bottom-(bottom-top)*k/4
            draw.line((left,y,right,y),fill="#dddddd",width=1)
    width=(right-left)/len(rows)*.55
    for i,(row,label,value) in enumerate(zip(rows,labels,vals)):
        x=left+(i+.5)*(right-left)/len(rows)
        if log:
            frac=(math.log10(value)-lo)/max(hi-lo,1e-9)
            y=bottom-frac*(bottom-top)
            draw.line((x-12,y,x+12,y),fill=color,width=4)
            draw.ellipse((x-6,y-6,x+6,y+6),fill=color,outline="white",width=1)
        else:
            frac=value/max(vmax,1e-12)
            y=bottom-frac*(bottom-top)
            draw.rectangle((x-width/2,y,x+width/2,bottom),fill=color)
        draw.text((x-42,bottom+12),label,fill=BLACK,font=font(16))
        draw.text((x-35,y-25),f"{value:.2g}",fill=BLACK,font=font(15))
    draw.rectangle((left,top,right,bottom),outline=BLACK,width=2)
    image.save(path)



def run_one(n, *, particles=60, inertia=.7298, patience=100, tolerance=1e-8, seed=2026, max_generations=1000,
capture=()):
    return ParticleSwarm(
        n, particles=particles, max_generations=max_generations, patience=patience,
        inertia=inertia, tolerance=tolerance, seed=seed,
    ).run(capture=capture)



def main(verbose=False):
    # Capture every step in the n=2 demonstration so any iteration can be inspected.
    demo=run_one(2,seed=247,patience=100,max_generations=1000,capture=tuple(range(1001)))
    visible=sorted(demo.snapshots)
    steps=[0, min(10,demo.generations), min(30,demo.generations), demo.generations]
    draw_contour(RESULTS/"function_population.png",demo.snapshots[demo.generations],demo.best_position)
    draw_surface(RESULTS/"surface_population.png",demo.snapshots[demo.generations],demo.best_position)
    draw_snapshots(demo,RESULTS/"search_steps.png",steps)


    dimension_rows=[]
    dimension_histories=[]
    dimension_colors={2:BLUE,3:RED,5:GREEN,10:"#9b59b6"}
    for n in (2,3,5,10):
        runs=[]
        for repeat in range(5):
            result=demo if n==2 and repeat==0 else run_one(n,seed=7000+n*100+repeat)
            runs.append(result)
            dimension_rows.append({"dimensions":n,"repeat":repeat+1,"best_value":result.best_value,
                "position_error":position_error(result.best_position),"generations":result.generations,
                "elapsed_seconds":result.elapsed_seconds,"stop_reason":result.stop_reason})
            if verbose:
                print(f"n={n},          запуск          {repeat+1}:          f={result.best_value:.4g},          ошибка
координат={position_error(result.best_position):.4g},  поколений={result.generations},  время={result.elapsed_seconds:.4f}
с")
        dimension_histories.append((f"n={n}",runs[0].best_history,dimension_colors[n]))
    with (RESULTS/"dimension_study.csv").open("w",encoding="utf-8",newline="") as stream:
        writer=csv.DictWriter(stream,fieldnames=list(dimension_rows[0]));                          writer.writeheader();
writer.writerows(dimension_rows)
    draw_line_chart(RESULTS/"convergence_dimensions.png","Сходимость PSO для разных размерностей",
        dimension_histories,y_label="Лучшее значение f(x)",x_label="Поколение",log_y=True)


    population_rows=[]
    for count in (20,40,80,120):
        runs=[run_one(2,particles=count,seed=8100+count*10+k,patience=80,max_generations=700) for k in range(5)]
        m,sd=mean_sd([r.best_value for r in runs]); tm,ts=mean_sd([r.elapsed_seconds for r in runs])
        gm,gs=mean_sd([r.generations for r in runs])
        population_rows.append({"particles":count,"mean_best":m,"sd_best":sd,"mean_generations":gm,
            "mean_seconds":tm,"sd_seconds":ts,"successes":sum(r.best_value<=1e-8 for r in runs),"runs":len(runs)})


    inertia_rows=[]
    for value in (.4,.7298,.9):
        runs=[run_one(2,inertia=value,seed=9100+int(value*1000)*10+k,patience=80,max_generations=700) for k in range(5)]
        m,sd=mean_sd([r.best_value for r in runs]); tm,ts=mean_sd([r.elapsed_seconds for r in runs])
        gm,gs=mean_sd([r.generations for r in runs])
        inertia_rows.append({"inertia":value,"mean_best":m,"sd_best":sd,"mean_generations":gm,
            "mean_seconds":tm,"sd_seconds":ts,"successes":sum(r.best_value<=1e-8 for r in runs),"runs":len(runs)})


    stopping_rows=[]
    for patience in (30,60,120):
```

```
        # Disable the target-value stop so patience alone controls termination.
        runs=[run_one(2,patience=patience,tolerance=0.0,seed=10100+patience*10+k,max_generations=700) for k in range(5)]
        m,sd=mean_sd([r.best_value for r in runs]); tm,ts=mean_sd([r.elapsed_seconds for r in runs])
        gm,gs=mean_sd([r.generations for r in runs])
        stopping_rows.append({"patience":patience,"mean_best":m,"sd_best":sd,"mean_generations":gm,
            "mean_seconds":tm,"sd_seconds":ts,"successes":sum(r.best_value<=1e-8 for r in runs),"runs":len(runs)})


    for filename,rows in (("population_study.csv",population_rows),("inertia_study.csv",inertia_rows),("stopping_study.csv",stopping_rows)):
        with (RESULTS/filename).open("w",encoding="utf-8",newline="") as stream:
            writer=csv.DictWriter(stream,fieldnames=list(rows[0]));writer.writeheader();writer.writerows(rows)
    draw_bars(RESULTS/"population_accuracy.png","Размер роя и среднее лучшее f(x)",population_rows,
        [str(r["particles"]) for r in population_rows],"mean_best",color=BLUE,log=True)
    draw_bars(RESULTS/"inertia_accuracy.png","Инерция и среднее лучшее f(x)",inertia_rows,
        [str(r["inertia"]) for r in inertia_rows],"mean_best",color=GREEN,log=True)
    draw_bars(RESULTS/"stopping_generations.png","Критерий остановки: среднее число поколений",stopping_rows,
        [str(r["patience"]) for r in stopping_rows],"mean_generations",color=RED)


    def aggregate(rows, key):
        result={}
        for n in (2,3,5,10):
            vals=[r[key] for r in rows if r["dimensions"]==n]
            result[str(n)]={"mean":statistics.mean(vals),"sd":statistics.stdev(vals)}
        return result
    summary={
        "variant":4,
        "objective":"sum(5*i*(x_i-5*i)^2), i=1..n",
        "reference_position_1based":[5*i for i in range(1,11)],
        "reference_minimum":0.0,
        "bounds":[-55.0,55.0],
        "interpretation":"The handout prints an unshifted formula but states x_i=5*i and calls the function moved. The
experiment uses the shifted formula consistent with the declared minimizer; the chosen bounds are stated in the report.",
        "base_parameters":{"particles":60,"max_generations":1000,"patience":100,"tolerance":1e-8,
          "inertia":.7298,"cognitive":1.49618,"social":1.49618,"velocity_limit_fraction":.2,"seed":247},

"demo":{"best_position":demo.best_position,"best_value":demo.best_value,"position_error":position_error(demo.best_positio
n),
          "generations":demo.generations,"elapsed_seconds":demo.elapsed_seconds,"stop_reason":demo.stop_reason,
          "best_history":demo.best_history,"mean_history":demo.mean_history,"snapshot_generations":steps,
          "snapshots":{str(g):demo.snapshots[g] for g in steps}},
        "dimension_summary":{"mean_best":aggregate(dimension_rows,"best_value"),
          "mean_position_error":aggregate(dimension_rows,"position_error"),
          "mean_generations":aggregate(dimension_rows,"generations"),
          "mean_seconds":aggregate(dimension_rows,"elapsed_seconds")},
        "dimension_runs":dimension_rows,"population_study":population_rows,
        "inertia_study":inertia_rows,"stopping_study":stopping_rows,
        "notes":{"mutation":"Mutation probability is not a PSO parameter; inertia weight is studied instead.",
          "random":"Uniform random coefficients in [0,1] are used, as in canonical PSO; the handout's mention of normal
distribution conflicts with its interval [0,1]."}
    }
    (RESULTS/"results_summary.json").write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding="utf-8")


    print("Лабораторная работа 7 — глобальный PSO, вариант 4")
    print("Цель: f(x)=Σ 5i·(xᵢ−5i)²; эталонный минимум 0 в точке (5,10,…,5n).")
    print("Демонстрационный запуск n=2:")
    print(f"  найдено: x=({demo.best_position[0]:.8f}, {demo.best_position[1]:.8f}), f={demo.best_value:.6g}")
    print(f"  генераций={demo.generations}, время={demo.elapsed_seconds:.4f} с, остановка: {demo.stop_reason}")
    print("\nСравнение размерностей (среднее по 5 запускам):")
    print("  n      f best       ошибка x    поколения   время, с")
    for n in (2,3,5,10):
        key=str(n)
        print(f"            {n:>2}                        {summary['dimension_summary']['mean_best'][key]['mean']:>12.4g}
{summary['dimension_summary']['mean_position_error'][key]['mean']:>11.4g}
{summary['dimension_summary']['mean_generations'][key]['mean']:>10.1f}
{summary['dimension_summary']['mean_seconds'][key]['mean']:>8.4f}")
    print("\nИсследование числа частиц:")
    for row in population_rows: print(f"  {row['particles']:>3} частиц: f̄={row['mean_best']:.4g} ± {row['sd_best']:.2g};
{row['mean_generations']:.1f}       поколений;       {row['mean_seconds']:.4f}       с;       точность       достигнута
{row['successes']}/{row['runs']}")
    print("\nИсследование инерции:")
    for row in inertia_rows: print(f"    w={row['inertia']:.4g}:  f̄={row['mean_best']:.4g}  ±  {row['sd_best']:.2g};
{row['mean_generations']:.1f} поколений; точность {row['successes']}/{row['runs']}")
    print("\nИсследование остановки:")
    for row in stopping_rows: print(f"          patience={row['patience']}:     f̄={row['mean_best']:.4g};
{row['mean_generations']:.1f} поколений; {row['mean_seconds']:.4f} с")
    if verbose:
        print("\nДинамика лучшего результата демонстрационного запуска:")
        for i,value in enumerate(demo.best_history):
            print(f"  поколение {i:4d}: f_best={value:.10g}; f_mean={demo.mean_history[i]:.10g}")
    print("\nГрафики и таблицы записаны в",RESULTS)



if __name__ == "__main__":
    if hasattr(sys.stdout, "reconfigure"):
```

```
    sys.stdout.reconfigure(encoding="utf-8")
parser=argparse.ArgumentParser(description="Воспроизводимый эксперимент PSO для лабораторной работы 7")
parser.add_argument("--verbose",action="store_true",help="вывести историю demo-запуска по поколениям")
main(parser.parse_args().verbose)
```